Artificial Intelligence

BS (CS) Spring 2026

Lab 04 Manual



Learning Objectives:

1. Uninformed searches
2. Breadth First Search
3. Depth First Search
4. Tree Search
5. Uniform Cost Search

Lab Manual

Uninformed Search:

Uninformed search is a class of general-purpose search algorithms. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called **blind** search. They operate in a brute force, meaning they try out every part of search space blindly.

Types:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first search
5. Uniform cost search

Breadth-First Search

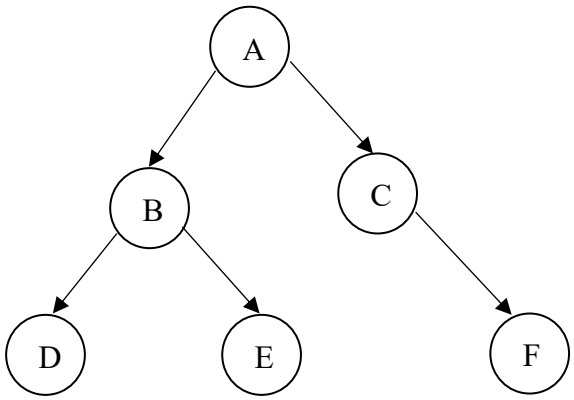
- BFS explores all the neighbor nodes at the present depth before moving on to the nodes at the next depth level.
  - It guarantees the shortest path from the starting node to the goal node in an unweighted graph.
  - It uses a queue data structure to keep track of the nodes to be explored next.

Step-by-Step Guide

1. Initialize an empty queue.
2. Enqueue the starting node and mark it as visited.
3. While the queue is not empty:
   a. Dequeue a node from the queue.
   b. Process the node (e.g., print or store it).
   c. Enqueue all unvisited adjacent nodes and mark them as visited.

Example

Consider the following graph:



Step-by-Step Guide

1. **Initialization**

- **Queue:** Empty Initially
- **Visited Nodes:** Start with only the source node marked as visited.

2. **Enqueue the starting node**
    - Enqueue node **A**
    - **Queue:** [A]
    - **Visited:** {A}

3. **Dequeue and process node A**
    - Dequeue **A** and process it.
    - Enqueue its unvisited neighbors: **B** and **C**.
    - **Queue:** [B, C]
    - **Visited:** {A, B, C}

4. **Dequeue and process node B**
    - Dequeue **B** and process it.
    - Enqueue its unvisited neighbors: **D** and **E**.
    - **Queue:** [C, D, E]
    - **Visited:** {A, B, C, D, E}

5. **Dequeue and process node C**
    - Dequeue **C** and process it.
    - Enqueue its unvisited neighbors: **F**.
    - **Queue:** [D, E, F]
    - **Visited:** {A, B, C, D, E, F}

6. **Process the remaining nodes**
    - Continue dequeuing and processing nodes (**D, E, F**) until the queue is empty.

Output:

Final traversal order:

A → B → C → D → E → F

Note: The queue ensures BFS processes nodes level by level. Newly discovered nodes are added to the queue, maintaining the breadth-first traversal order. This makes BFS effective for finding the shortest path in unweighted graphs and exhaustively exploring all possibilities.

Implementation

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            queue.extend(graph[node])

print("BFS Traversal:")
bfs(graph, 'A')
```

```
BFS Traversal:
A B C D E F
```

BFS in Real-World AI Applications

1. Network Broadcasting
2. Peer to peer networks
3. Medical Diagonsis
4. AI in games (in which it require minimal moves)
5. Pathfinding Algorithms (Google maps)
6. Robot navigations.

Depth-First Search

Depth First Search (DFS) is an algorithm used for traversing or searching tree and graph data structures. It explores each branch of a graph or tree as deeply as possible before backtracking to examine unexplored branches. This depth-first approach makes DFS particularly useful for solving problems where a solution is found by exploring paths one at a time.

- DFS explores as far as possible along each branch before backtracking.
- It may not find the shortest path to the goal node.
- It uses a stack data structure (or recursion) to keep track of the nodes to be explored next.

Step-by-Step Guide

1. **Initialization**
   - Mark All Nodes as Unvisited: Start with a list or a dictionary to track visited nodes.
   - Select a Starting Node: Choose the node from which the traversal begins.

2. **Traversal Process**
    - Visit the Starting Node:
        - Mark it as visited.
        - Add it to a stack (or recursive call stack) to keep track of traversal.
    - Explore Adjacent Nodes:
        - For each unvisited neighbor of the current node:
            - Visit the node.
            - Mark it as visited.
            - Add it to the stack.
    - Backtrack When Necessary:
        - If a node has no unvisited neighbors, backtrack to the previous node in the stack and continue.

3. **End Condition**

- Continue the process until all nodes in the graphs have been visited.

Different approaches of DFS

1. **Iterative DFS (Using Stack)**
    1. Push the starting node onto the stack.
    2. While the stack is not empty:
        a. Pop the top node.
        b. Mark it as visited.
        c. Push all unvisited neighbors of the popped node onto the stack.

2. **Recursive DFS**

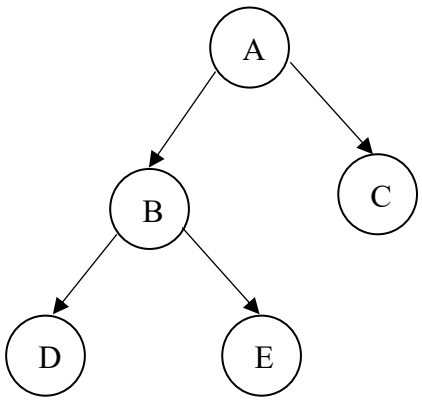
1. Start with the first node.
2. For each unvisited neighbor of the current node:

- Recursively visit the neighbor.

3. Backtrack once all neighbors are visited.

Example

Consider the following graph:



- **Step 1:** Start at A (mark visited).
- **Step 2:** Visit B (mark visited and add its neighbors D and E to the stack).
- **Step 3:** Visit D (mark visited, no neighbors left, backtrack to B).
- **Step 4:** Visit E (mark visited, no neighbors left, backtrack to A).
- **Step 5:** Visit C (mark visited).

Pseudocode (Recusive DFS)

```
function DFS(node):
    mark node as visited
    for each neighbor of node:
        if neighbor is not visited:
            DFS(neighbor)
```

Pseudocode (Iterative DFS)

```
function DFS_Iterative(startNode):
    create a stack and push startNode onto it
    while stack is not empty:
        currentNode = pop from stack
        if currentNode is not visited:
            mark currentNode as visited
            for each neighbor of currentNode:
                if neighbor is not visited:
                    push neighbor onto stack
```

Comparison Table

Recursive DFS	Iterative DFS
Simplifies implementation but uses the call stack, which can lead to stack overflow in deep or large graphs.	Avoid stack overflow by using an explicit stack, making it more suitable for deep graphs.

DFS in Real-World AI Applications

1. Cycle Detection
- In network analysis, detecting feedback loops or circular dependencies.
2. Pathfinding
- Solving puzzles like mazes or determining routes in navigation systems
3. Topological Sorting
- Task scheduling where certain tasks must be completed before others.
4. Connected Components
- In social networks, finding clusters or groups of users connected by friendships.
5. Solving puzzles and games (chess, Sudoku)
6. Strongly connected components
- Analyzing web pages to determine clusters of interlinked pages.

Comparison of BFS and DFS

Criteria	Breadth-First Search	Depth First Search
Approach	Explore node level by level	Explore nodes depth wise
Data Structure	Queue	Stack (Recursion)
Shortest Path	Yes (in weighted graphs)	No
Memory Usage	Higher (Store all neighbors)	Lower (only store current path)
Performance	Suitable for shallow graphs	Efficient for deep and complex graphs
Time Complexity	$0\left(b^{d+1}\right)$	$0\left(b^{m}\right)$
Space Complexity	$O(b^{d+1})$	$O(bm)$
Usage	Better for shortes path	Better for deep searches

Tree Search

- Tree search is a general framework that combines a search strategy (like BFS or DFS) with a data structure to explore the search space.
- It can be used with informed search algorithms by incorporating heuristics to guide the search.
- The choice of search strategy and data structure determines the efficiency and effectiveness of the search algorithm.

Example

```python
from collections import deque

class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []

def shortest_path_tree_search(root, target):
    # Initialize a queue for BFS traversal
    queue = deque([(root, [root])])

    # Perform BFS traversal
    while queue:
        node, path = queue.popleft()
        if node.value == target:
            return path
        for child in node.children:
            queue.append((child, path + [child.value]))

# Example tree structure
#        A
#      / | \
#     B  C  D
#    / \    |
#   E   F   G
root = TreeNode('A')
root.children = [TreeNode('B'), TreeNode('C'), TreeNode('D')]
root.children[0].children = [TreeNode('E'), TreeNode('F')]
root.children[2].children = [TreeNode('G')]
```

```python
# Find the shortest path from node 'A' to node 'G' using tree search (BFS)
shortest_path = shortest_path_tree_search(root, 'G')
print("Shortest Path (Tree Search):", shortest_path)
```

```
Shortest Path (Tree Search): [<__main__.TreeNode object at 0x7bca43309420>, 'D', 'G']
```

Uniform Cost Search

Uniform Cost Search (UCS) is a priority-based search algorithm that **systematically explores a graph or tree** to find the path with the **minimum cumulative cost.** UCS uses a **priority queue** to ensure that nodes with the lowest path cost are expanded first, making it particularly suited for scenarios where paths have varying costs.

Features:

- Guarantees **optimal solution** if all costs are positive.
- Uses a **priority queue** to expand the lowest-cost node first.
- Works well in weighted graphs.

Step-by-Step Guide:

1. **Initialize** a priority queue (frontier) that orders nodes by their total path cost.
2. **Insert the start node** into the priority queue with path cost = 0.

3. Initialize an **empty explored set** to keep track of nodes that have already been expanded.
4. Repeat while the priority queue is not empty:
    a. Remove the node with the **lowest path cost** from the queue.
5. Check if the current node is the goal:
    a. If yes, stop and return the solution path and total cost.
6. Mark the current node as explored.
7. Expand the current node by generating all its neighboring nodes.
8. For each neighbor:
    a. Calculate the **new cumulative path cost**.
    b. If the neighbor is **not explored** or a **cheaper path** to it is found:
        - Update its cost.
        - Record the current node as its parent.
        - Insert the neighbor into the priority queue with priority equal to the new cost.
9. If the priority queue becomes empty and the goal has not been found:
    - **Report failure** (no solution exists).

Pseudocode

```
def Uniform_Cost_Search(start,goal):
  priority_queue = empty
  priority_queue.add(start, cost = 0, path = [start]) #insert into
priority queue
  visited = set()      #empty set initially

  #loop to iterate through the priority queue
  while priority_queue is not empty:
    #remove the node with the minimum cost from the priority queue
    priority_queue.remove(current_node, current_cost, current_path);

    if current_node == goal:
      return (current_path, current_cost);
    if current_node not in visited:
      visited.add(current_node);
      for neighbor in current_node:
        path__cost = current_cost + cost(current_node, neighbor)
        new_path = current_path + neighbor

        priority_queue.add(neighbor, path__cost, new_path)
  return ("Goal not reachable :( ")
```

UCS in Real-World AI Applications

1. Pathfinding in Robotics
2. Autonomous Navigation Systems
3. Automated Delivery System
4. Resource Management Systems

Note: UCS is **inefficiency in large or complex graphs** without an informed heuristic.